https://hjfy.top/arxiv/2203.03605

模型名称：端到端目标检测的改进降噪锚框 DETR

Hao Zhang[1,3*†], Feng Li[1,3*†], Shilong Liu[2,3*†], Lei Zhang[3‡], Hang Su[2], Jun Zhu[2], Lionel M. Ni[1,4], Heung-Yeung Shum[1,3]

[1]The Hong Kong University of Science and Technology.
[2]Dept. of CST., BNRist Center, Institute for AI, Tsinghua University.
[3]International Digital Economy Academy (IDEA).
[4]The Hong Kong University of Science and Technology (Guangzhou).
{hzhangcx,fliay}@connect.ust.hk
{liusl20}@mails.tsinghua.edu.cn
{suhangss,dcszj}@mail.tsinghua.edu.cn
{ni,hshum}@ust.hk
{leizhang}@idea.edu.cn

摘要 我们提出 DINO (**D**ETR with **I**mproved de**N**oising anch**O**r boxes)，一种最先进的端到端目标检测器。DINO 通过使用对比方式的降噪训练、用于锚点初始化的混合查询选择方法以及用于框预测的向前两次方案，在性能和效率上优于之前的 DETR 类模型。DINO 在 COCO 数据集上使用 ResNet-50 主干网络和多尺度特征，在 49.4 轮次中达到 12 AP，在 51.3 轮次中达到 24 AP，分别比之前的最佳 DETR 类模型 DN-DETR 提高了 **+6.0 AP** 和 **+2.7 AP**。DINO 在模型大小和数据大小方面都能良好扩展。无需额外技巧，在 Objects365 数据集上使用 SwinL 主干进行预训练后，DINO 在 COCO val2017 (**63.2 AP**) 和 test-dev (**63.3 AP**) 上取得了最佳结果。与排行榜上的其他模型相比，DINO 在实现更好结果的同时显著减少了模型大小和预训练数据大小。我们的代码将在 https://github.com/IDEACVR/DINO 发布。

Keywords: Object Detection; Detection Transformer; End-to-End Detector

---

* 同等贡献。列出顺序是随机的。
† 这项工作是在张浩、李峰和刘世龙在 IDEA 实习期间完成的。
‡ 通讯作者。

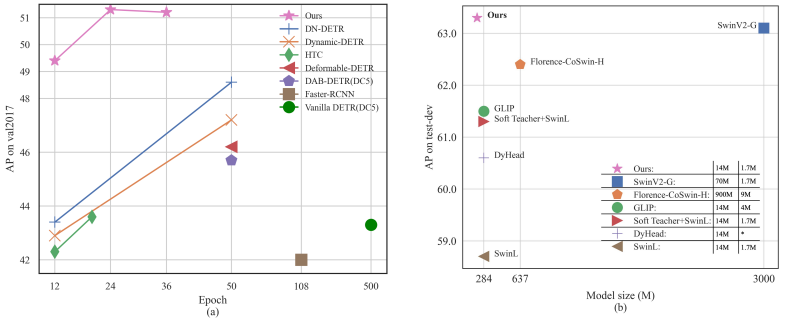

图 1. COCO 上的 AP 与其他检测模型的比较。(a) 与具有 ResNet-50 主干网络的模型在训练轮次方面的比较。标记为 DC5 的模型使用了膨胀的更大分辨率特征图。其他模型使用多尺度特征。(b) 与最先进模型在预训练数据大小和模型大小方面的比较。最先进模型来自 COCO test-dev 排行榜。在图例中，我们列出了主干预训练数据大小（第一个数字）和检测预训练数据大小（第二个数字）。∗ 表示数据大小未公开。

1 引言

目标检测是计算机视觉中的一个基础任务。经典的基于卷积的目标检测算法 [31,35,19,2,12] 已取得了显著的进展。尽管这类算法通常包括手工设计的组件，如锚点生成和非极大值抑制（NMS），但它们产生了最佳的检测模型，如 DyHead [7]、Swin [23] 和 SwinV2 [22]，以及 HTC++ [4]，这在 COCO test-dev 排行榜上得到了证据 [1]。

与传统检测算法不同，DETR [3] 是一种基于 Transformer 的新型检测算法。它消除了对手工设计组件的需求，并且在与优化的传统检测器如 Faster RCNN [31] 相当的性能上取得了成功。不同于之前的检测器，DETR 将目标检测建模为一个集合预测任务，并通过二分图匹配分配标签。它利用可学习的查询来探测对象的存在，并结合图像特征图中的特征，这类似于软 ROI 池化 [21]。

尽管其表现令人期待，DETR 的训练收敛速度较慢且查询的含义不明确。为了解决这些问题，提出了许多方法，例如引入可变形注意力 [41]，解耦位置和内容信息 [25]，提供空间先验 [11,39,37] 等。最近，DAB-DETR [21] 提出将 DETR 的查询形式化为动态锚框（DAB），这弥合了传统基于锚框的检测器与 DETR 类检测器之间的差距。DN-DETR [17] 通过引入降噪（DN）技术进一步解决了二分匹配的不稳定性。DAB 和 DN 的结合使 DETR 类模型在训练效率和推理性能方面都具有竞争力。

如今最好的检测模型基于改进的经典检测器，如 DyHead [8] 和 HTC [4]。例如，SwinV2 [22] 中展示的最佳结果是使用 HTC++ [4,23] 框架训练的。两个主要原因导致了这一现象：1) *之前的 DETR 类模型不如改进的经典检测器*。大多数经典检测器已经得到了充分的研究和高度优化，与新开发的 DETR 类模型相比，性能更好。例如，目前表现最好的 DETR 类模型在 COCO 上的 AP 仍然低于 50 。2) DETR 类模型的 *可扩展性*尚未得到充分研究。关于 DETR 类模型在扩展到大骨干网络和大规模数据集时的表现，还没有报告结果。本文旨在解决这两个问题。

具体来说，通过改进降噪训练、查询初始化和框预测，我们设计了一个基于 DN-DETR [17]、DAB-DETR [21] 和可变形 DETR [41] 的新 DETR 类似模型。我们将我们的模型命名为 **DINO** (带改进降噪锚点或框的 DETR)。如图1所示，COCO 上的比较显示了 DINO 的优越性能。特别是，DINO 表现出强大的可扩展性，在 COCO test-dev 排行榜上创下了 63.3 AP 的新纪录 [1]。

作为 DETR 类似的模型，DINO 包含一个主干网络、一个多层 Transformer 编码器、一个多层 Transformer 解码器以及多个预测头。遵循 DAB-DETR [21]，我们将解码器中的查询表述为动态锚框，并在解码器层中逐步优化它们。遵循 DN-DETR [17]，我们在 Transformer 解码器层中加入带有噪声的真实值标签和框，以帮助在训练期间稳定二分匹配。我们还采用可变形注意力 [41] 以提高计算效率。此外，我们提出了三种新方法，如下所示。首先，为了改进一对一匹配，我们通过同时添加相同真实值的正负样本提出了*一种对比降噪训练*。在同一个真实值框上添加两种不同的噪声后，我们将噪声较小的框标记为正例，另一个标记为负例。对比降噪训练有助于模型避免对同一目标的重复输出。其次，查询的动态锚框表述将 DETR 类似的模型与经典的两阶段模型联系起来。因此我们提出了一种*混合查询选择方法*，有助于更好地初始化查询。我们从编码器的输出中选择初始锚框作为位置查询，类似于 [41,39]。然而，我们仍然让内容查询可以学习，鼓励第一个解码器层关注空间先验。第三，为了利用后续层的精炼框信息来帮助优化其相邻早期层的参数，我们提出了一种新的*向前看两次方案*，用后续层的梯度来校正更新后的参数。

我们通过在 COCO [20] 检测基准上的广泛实验来验证 DINO 的有效性。如图1所示，DINO 在 12 轮次中达到 49.4 AP，在 24 轮次中达到 51.3 AP，使用 ResNet-50 和多尺度特征，分别比之前的最佳 DETR 类模型提高

了 **+6.0** AP 和 **+2.7** AP。此外，DINO 在模型大小和数据大小方面都能良好扩展。在 Objects365 [33] 数据集上使用 SwinL [23] 主干进行预训练后，DINO 在 COCO val2017 (**63.2** AP) 和 test-dev (**63.3** AP) 基准测试中取得了最佳结果，如表3所示。与排行榜 [1] 上的其他模型相比，我们的模型大小减少了 **1/15**，相比 SwinV2-G [22]。与 Florence [40] 相比，我们的预训练检测数据集减少到 **1/5**，主干预训练数据集减少到 **1/60**，同时取得了更好的结果。

我们将我们的贡献总结如下。

1. 我们设计了一种新的端到端 DETR 类似的目标检测器，包括对比性 DN 训练、混合查询选择以及针对 DINO 模型不同部分的两次前瞻。
2. 我们进行了大量的消融实验以验证 DINO 中不同设计选择的有效性。结果表明，DINO 在 49.4 有效轮次和 12 轮次中分别达到了 51.3 AP 和 24 AP，使用 ResNet-50 和多尺度特征，显著优于之前的最佳 DETR 类模型。特别是，在 12 轮次中训练的 DINO 在小物体上表现出更显著的提升，提升了 **+7.5** AP。
3. 我们表明，不使用任何花招，DINO 可以在公开基准上达到最佳性能。在 Objects365 [33] 数据集上使用 SwinL [23] 骨干网络进行预训练后，DINO 在 COCO val2017 (**63.2** AP) 和 test-dev (**63.3** AP) 基准测试中都取得了最佳结果。据我们所知，这是第一次端到端的 Transformer 检测器在 COCO 排行榜上超越了最先进的 (SOTA) 模型 [1]。

2 相关工作

2.1 经典目标检测器

基于卷积的早期目标检测器要么是两阶段模型，要么是一阶段模型，基于手工设计的锚框或参考点。两阶段模型 [30,13] 通常使用区域建议网络（RPN） [30] 来提出潜在的框，然后在第二阶段进行细化。一阶段模型如 YOLO v2 [28] 和 YOLO v3 [29] 直接输出相对于预定义锚框的偏移量。最近，一些基于卷积的模型如 HTC++ [4] 和 Dyhead [7] 在 COCO 2017 数据集 [20] 上取得了顶级性能。然而，基于卷积的模型的性能依赖于它们生成锚框的方式。此外，它们需要手工设计的组件如 NMS 来去除重复的框，因此无法进行端到端的最优化。

2.2 DETR 及其变体

Carion *et al.* [3] 提出了一种基于 Transformer 的端到端目标检测器，名为 DETR（DEtection TRansformer），没有使用像锚点设计和 NMS 这样的手工设计组件。许多后续论文尝试解决由解码器交叉注意力引入的 DETR 训练收敛慢的问题。例如，Sun *et al.* [34] 设计了一个仅使用编码器的 DETR，而没有使用解码器。Dai *et al.* [7] 提出了一种动态解码器，以从多个特征层中关注重要区域。

另一类工作是深入理解 DETR 中的解码器查询。许多论文从不同角度将查询与空间位置相关联。可变形 DETR [41] 预测 2 D 锚点，并设计了一个可变形注意力模块，该模块仅关注参考点周围的某些采样点。高效 DETR [39] 从编码器的稠密预测中选择前 K 个位置以增强解码器查询。DAB-DETR [21] 进一步将 2 D 锚点扩展为 4 D 锚框坐标来表示查询，并在每个解码器层中动态更新框。最近，DN-DETR [17] 引入了一种降噪训练方法以加速 DETR 训练。它将添加噪声的真值标签和框输入解码器，并训练模型以重建原始内容。本文中我们的 DINO 工作基于 DAB-DETR 和 DN-DETR，并且也采用可变形注意力以提高计算效率。

2.3 目标检测的大规模预训练

大规模预训练对自然语言处理 [10] 和计算机视觉 [27] 都产生了重大影响。目前最佳性能的检测器大多是通过在大规模数据上预训练的大规模主干网络实现的。例如，Swin V2 [22] 将其主干大小扩展到 3.0 十亿参数，并使用 70 M 私人收集的图像进行模型预训练。Florence [40] 首先使用 900 M 私人整理的图像-文本对预训练其主干，然后使用带有标注或伪框的 9 M 图像预训练其检测器。相比之下，DINO 仅使用公开的 SwinL [23] 主干和公共数据集 Objects365 [33]（1.7M 标注图像）就达到了最先进结果。

3 DINO: DETR 带有 Improved DeNoising Anchor Boxes

3.1 初步研究

如在 Conditional DETR [25] 和 DAB-DETR [21] 中所研究的，显然 DETR [3] 中的查询由两部分组成：位置部分和内容部分，本文将它们分别称为位置查询和内容查询。DAB-DETR [21] 明确地将 DETR 中的每个位置

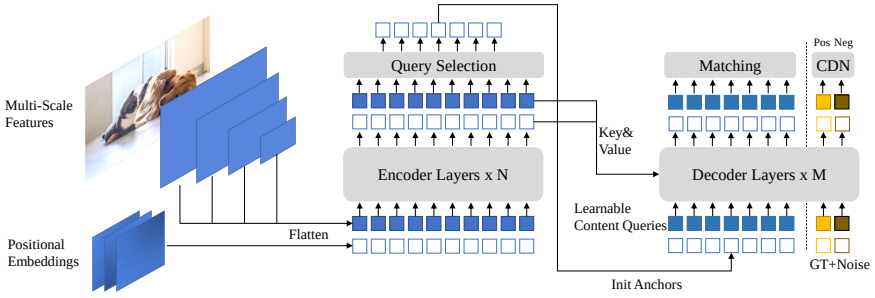

图 2. 我们提出的 DINO 模型的框架。我们的改进主要在 Transformer 编码器和解码器中。选择最后一层的 Top-K 编码器特征来初始化 Transformer 解码器的位置查询，而内容查询则保持为可学习参数。我们的解码器还包含一个带有正例和负例的对比降噪（CDN）部分。

查询表示为一个 4D 锚框 (x, y, w, h) ，其中 x 和 y 是该框的中心坐标，而 w 和 h 对应于其宽度和高度。这种显式的锚框表述使得在解码器中逐层动态优化锚框变得容易。

DN-DETR [17] 引入了一种降噪（DN）训练方法，以加速类似 DETR 模型的训练收敛。它表明 DETR 中的收敛缓慢问题是由二分匹配的不稳定性引起的。为了解决这个问题，DN-DETR 提出将带有噪声的真实标签（GT）和边界框额外输入到 Transformer 解码器中，并训练模型来重建真实标签。添加的噪声 $(\Delta x, \Delta y, \Delta w, \Delta h)$ 受 $|\Delta x| < \frac{\lambda w}{2}$ 、$|\Delta y| < \frac{\lambda h}{2}$ 、$|\Delta w| < \lambda w$ 和 $|\Delta y| < \lambda h$ 的约束，其中 (x, y, w, h) 表示一个真实边界框，λ [1] 是一个超参数，用于控制噪声的规模。由于 DN-DETR 遵循 DAB-DETR 将解码器查询视为锚点，因此带有噪声的真实标签框可以被视为一个特殊的锚点，其中真实标签框附近为 λ 通常较小。除了原始 DETR 的查询外，DN-DETR 添加了一个 DN 部分，将带有噪声的真实标签和框输入到解码器中，以提供辅助的 DN 损失。DN 损失有效地稳定并加速了 DETR 的训练，并可以插入到任何类似 DETR 的模型中。

可变形 DETR [41] 是另一项早期工作，用于加快 DETR 的收敛速度。为了计算可变形注意力，它引入了参考点的概念，使得可变形注意力可以关注参考点周围的少量关键采样点。参考点的概念使得可以开发几种技术来进

[1] DN-DETR 论文 [17] 使用 λ_1 和 λ_2 表示中心偏移和框缩放的噪声尺度，但将 $\lambda_1 = \lambda_2$ 设为。在本文中，为了简化，我们使用 λ 替代 $\lambda_1 \text{and} \lambda_2$ 。

一步提升 DETR 的性能。第一种技术是查询选择[2]，其中选择编码器中的特征和参考框作为输入直接传递给解码器。第二种技术是在两个解码器层之间采用仔细设计的梯度分离进行迭代边界框优化。我们在论文中将这种梯度分离技术称为“向前看一次”。

遵循 DAB-DETR 和 DN-DETR，DINO 将位置查询建模为动态锚框，并通过额外的 DN 损失进行训练。需要注意的是，DN-DETR 也采用了来自可变形 DETR 的几种技术以获得更好的性能，包括其可变形注意力机制以及在层参数更新中的“向前看一次”实现。DINO 进一步采用了来自可变形 DETR 的查询选择思想，以更好地初始化位置查询。在此强大的基准之上，DINO 引入了三种新方法以进一步提升检测性能，分别将在第3.3节、第3.4节和第3.5节中描述。

3.2 模型概述

作为 DETR 类似的模型，DINO 是一个端到端架构，包含一个主干网络、多层 Transformer [36] 编码器、多层 Transformer 解码器和多个预测头。整体流水线如图2所示。给定一张图像，我们使用类似 ResNet [14] 或 Swin Transformer [23] 的主干网络提取多尺度特征，然后将它们与相应的位置嵌入一起输入 Transformer 编码器。在编码器层进行特征增强后，我们提出了一种新的混合查询选择策略，以初始化锚点作为解码器的位置查询。请注意，这种策略不初始化内容查询，而是让它们可学习。混合查询选择的更多细节见第3.4节。通过初始化的锚点和可学习的内容查询，我们使用可变形注意力 [41] 将编码器输出的特征结合起来，并逐层更新查询。最终输出由精炼的锚框和由精炼内容特征预测的分类结果组成。如同 DN-DETR [17] 中所述，我们有一个额外的 DN 分支来进行降噪训练。除了标准的 DN 方法外，我们提出了一种新的对比降噪训练方法，通过考虑困难负例，将在第3.3节中介绍。为了充分利用后续层中的精炼框信息来帮助优化相邻早期层的参数，提出了一种新颖的两次向前方法，在相邻层之间传递梯度，这将在第3.5节中描述。

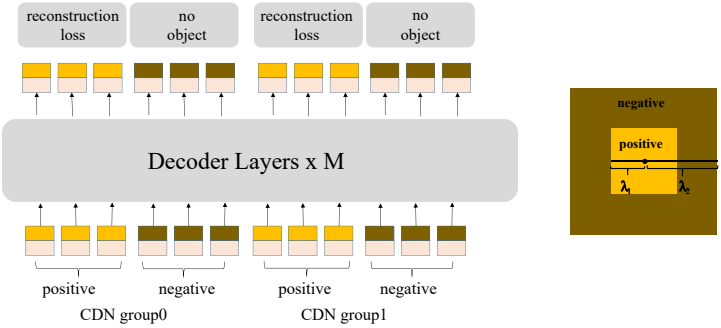

图 3. CDN 组的结构及其正负样本示例。尽管正负样本都是可以表示为 4 D 空间中点的 4 D 锚点，我们为了简化，将它们表示为同心方阵中的点在 2 D 空间中。假设方阵中心是一个 GT 框，内方阵内部的点被视为正样本，而位于内方阵和外方阵之间的点则被视为负样本。

3.3 对比降噪训练

DN-DETR 在稳定训练和加速收敛方面非常有效。借助 DN 查询，它学习根据附近有 GT 框的锚点进行预测。然而，它缺乏对附近没有物体的锚点进行“无物体”预测的能力。为了解决这个问题，我们提出了一种对比降噪（CDN）方法来 * 拒绝 * 无用的锚点。

实现： DN-DETR 有一个超参数 λ 用于控制噪声尺度。生成的噪声不超过 λ ，因为 DN-DETR 希望模型从适度噪声的查询中重构真实值（GT）。在我们的方法中，我们有两个超参数 λ_1 和 λ_2 ，其中 $\lambda_1 < \lambda_2$ 。如图 3 中的同心方阵所示，我们生成两种类型的 CDN 查询：正例查询和负例查询。位于内方阵内的正例查询的噪声尺度小于 λ_1 ，并期望重构其对应的真实值框。位于内方阵和外方阵之间的负例查询的噪声尺度大于 λ_1 且小于 λ_2 。它们期望预测“无对象”。我们通常采用较小的 λ_2 ，因为更接近真实值框的困难负例更有助于提高性能。如图 3 所示，每个 CDN 组都有一组正例查询和负例查询。如果一张图像有 n 个真实值框，一个 CDN 组将有 $2 \times n$ 个查询，每个真实值框生成一个正例和一个负例查询。类似于 DN-DETR，我们也使用多个 CDN 组来提高方法的有效性。重构损失为 l_1 和 GIOU 损失用于框回归，以及焦点损失 [19] 用于分类。将负例分类为背景的损失也是焦点损失。

分析： 我们的方法有效的原因在于它能够抑制混淆，并为预测边界框选择高质量的锚点（查询）。当多个锚点接近一个目标时会发生混淆。在这种情况

[2] 也被称为“两阶段”在可变形 DETR 论文中。由于“两阶段”这一名称可能与经典检测器混淆，我们在论文中使用“查询选择”这一术语。

下，模型很难决定选择哪个锚点。混淆可能导致两个问题。第一个是重复预测。尽管 DETR 类模型可以通过基于集合的损失和自注意力 [3] 来抑制重复框，但这种能力是有限的。如图 8 左侧所示，当用 DN 查询替换我们的 CDN 查询时，箭头指向的男孩有 3 个重复预测。使用 CDN 查询，我们的模型可以区分锚点之间的微小差异，并避免重复预测，如图 8 右侧所示。第二个问题是可能选择一个远离真实边界框的不需要的锚点。虽然降噪训练 [17] 已经提高了模型选择附近锚点的能力，但 CDN 通过教模型拒绝较远的锚点进一步提高了这种能力。

有效性： 为了展示 CDN 的有效性，我们定义了一个称为平均 Top-K 距离 (ATD(k)) 的指标，并用它来评估匹配部分中锚点与目标 GT 框之间的距离。如同 DETR 一样，每个锚点对应一个预测结果，该结果可能与 GT 框或背景匹配。我们仅考虑那些与 GT 框匹配的情况。假设我们在验证集中有 N 个 GT 边界框 $b_0, b_2, ..., b_{N-1}$ ，其中 $b_i = (x_i, y_i, w_i, h_i)$ 。对于每个 b_i ，我们可以找到其对应的锚点，并将其表示为 $a_i = (x'_i, y'_i, w'_i, h'_i)$ 。a_i 是解码器的初始锚点框，在匹配过程中，经过最后一层解码器精炼后的框被分配给 b_i 。然后我们有

$$ATD(k) = \frac{1}{k} \sum \{topK\left(\{\|b_0 - a_0\|_1, \|b_1 - a_1\|_1, ..., \|b_{N-1} - a_{N-1}\|_1\}, k\right)\} \tag{1}$$

其中 $\|b_i - a_i\|_1$ 是 l_1 之间的 b_i 和 a_i 距离，$topK(\mathbf{x}, k)$ 是返回 k 个 $\mathbf{x}$ 中最大元素的集合的函数。我们选择前 K 个元素的原因是，当 GT 框与较远的锚框匹配时，更容易发生混淆问题。如图 4 (a) 和 (b) 所示，DN 在整体上已经足够好地选择了好的锚框。然而，CDN 对于小物体找到了更好的锚框。图 4 (c) 显示，在 ResNet-50 和多尺度特征下，CDN 查询在 12 轮次中使小物体的 +1.3 AP 相比 DN 查询有所提高。

3.4 混合查询选择

在 DETR [3] 和 DN-DETR [17] 中，解码器查询是静态嵌入，不从单个图像中获取任何编码器特征，如图5 (a) 所示。它们直接从训练数据中学习锚点（在 DN-DETR 和 DAB-DETR 中）或位置查询（在 DETR 中），并将内容查询设置为所有 0 向量。可变形 DETR [41] 同时学习位置和内容查询，这是静态查询初始化的另一种实现方式。为了进一步提高性能，可变形 DETR [41] 有一个查询选择变体（在 [41] 中称为“两阶段”），它从最后一个编码器层中选择前 K 个编码器特征作为先验，以增强解码器查询。如图5

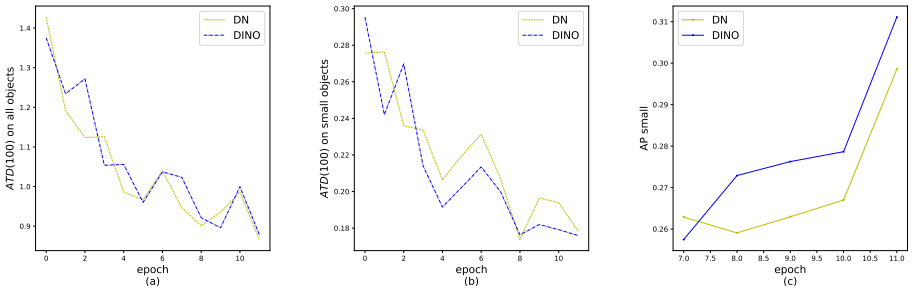

图 4. (a) 和 (b) 分别是关于所有对象和小对象的 $ATD(100)$。(c) 小对象上的平均精度 (AP)。

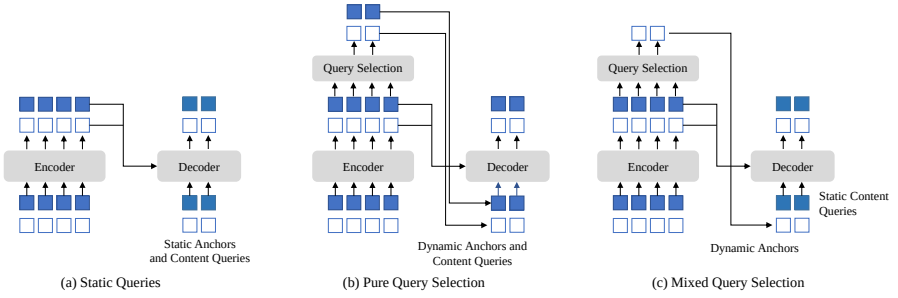

图 5. 三种不同查询初始化方法的比较。术语“静态”表示它们在不同图像的推理中保持相同。这些静态查询的一种常见实现方式是让它们可学习。

(b) 所示，位置和内容查询均由所选特征的线性变换生成。此外，这些选定的特征被输入到辅助检测头以获得预测框，这些框用于初始化参考框。同样，高效 DETR [39] 也根据每个编码器特征的对象性（类别）得分选择前 K 个特征。

我们的模型中查询的动态 4 D 锚框公式使其与解码器位置查询密切相关，可以通过查询选择进行改进。我们遵循上述做法，并提出一种混合查询选择方法。如图 5 (c) 所示，我们仅使用与选定的前 K 个特征相关的位置信息来初始化锚框，但保持内容查询不变。需要注意的是，Deformable DETR [41] 使用前 K 个特征不仅增强了位置查询，还增强了内容查询。由于所选特征是未经进一步优化的初步内容特征，它们可能会对解码器产生歧义和误导。例如，一个选定的特征可能包含多个对象或只是对象的一部分。相比之下，我们的混合查询选择方法仅用前 K 个选定特征增强位置查询，并保持内容查询可学习如前。这有助于模型使用更好的位置信息从编码器中提取更全面的内容特征。

3.5 展望两次

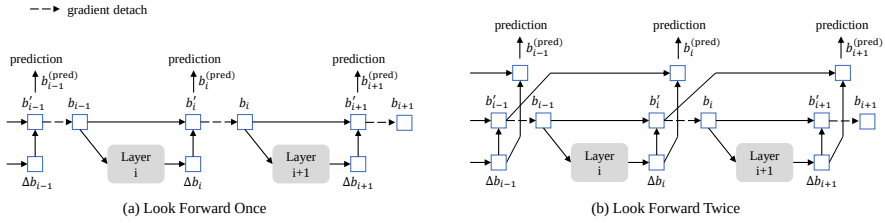

图 6. 可变形 DETR 与我们方法的框更新比较。

我们在本节中提出了一种新的框预测方法。可变形 DETR 中的迭代框细化 [41] 阻止梯度反向传播以稳定训练。我们将该方法命名为“向前看一次”，因为层 i 的参数仅基于框 b_i 的辅助损失进行更新，如图 6 (a) 所示。然而，我们推测来自后续层的改进框信息可能更有助于纠正其相邻早期层的框预测。因此我们提出另一种称为“向前看两次”的框更新方法，其中层-i 的参数受到层-i 和层-$(i+1)$ 损失的影响，如图 6 (b) 所示。对于每个预测的偏移量 Δb_i ，它将用于更新框两次，一次用于 b'_i ，另一次用于 $b_{i+1}^{(pred)}$ ，因此我们将我们的方法命名为“向前看两次”。

预测框 $b_i^{(\text{pred})}$ 的最终准确率由两个因子决定：初始框 b_{i-1} 的质量和框 Δb_i 的预测偏移量。一次向前方案仅优化后者，因为梯度信息从层-i 脱离到层-$(i-1)$ 。相反，我们改进了初始框 b_{i-1} 和预测框偏移量 Δb_i 。提高质量的一种简单方法是用下一层 Δb_{i+1} 的输出来监督层 i 的最终框 b'_i 。因此，我们将 b'_i 和 Δb_{i+1} 的和作为层-$(i+1)$ 的预测框。

更具体地说，给定第 i 层的输入框 b_{i-1} ，我们通过以下方式得到最终预测框 $b_i^{(\text{pred})}$ ：

$$
\begin{aligned}
\Delta b_i &= \text{Layer}_\text{i}(b_{i-1}), & b'_i &= \text{Update}(b_{i-1}, \Delta b_i), \\
b_i &= \text{Detach}(b'_i), & b_i^{(\text{pred})} &= \text{Update}(b'_{i-1}, \Delta b_i),
\end{aligned}
\tag{2}
$$

其中 b'_i 是 b_i 的未分离版本。术语 Update$(\cdot,\cdot)$ 是一个通过预测的框偏移量 Δb_i 来细化框 b_{i-1} 的函数。我们采用相同的方法进行框更新[3] 如 Deformable DETR [41] 所示。

4 实验

4.1 设置

数据集和主干网络: 我们在 COCO 2017 目标检测数据集 [20] 上进行评估，该数据集分为 train2017 和 val2017（也称为 minival）。我们报告使用两种不同主干网络的结果：在 ImageNet-1k [9] 上预训练的 ResNet-50 [14] 和在 ImageNet-22k [9] 上预训练的 SwinL [23]。DINO 使用 ResNet-50 在 train2017 上进行训练，没有使用额外数据，而 DINO 使用 SwinL 首先在 Object365 [33] 上预训练，然后在 train2017 上微调。我们在不同 IoU 阈值和物体尺度下报告 val2017 上的标准平均准确率（AP）结果。我们还报告了使用 SwinL 的 DINO 的 test-dev 结果。

实现细节: DINO 包括一个主干网络、一个 Transformer 编码器、一个 Transformer 解码器和多个预测头。在附录 D 中，我们提供了更多的实现细节，包括所有用于我们的模型的超参数和工程技巧，以便于那些希望复现我们结果的人。我们将在盲审后发布代码。

4.2 主要结果

12 轮次情景: 通过我们改进的锚框降噪和训练损失，训练过程可以显著加速。如表 1所示，我们将我们的方法与强基准方法进行了比较，包括基于卷积的方法 [30,4,7] 和 DETR 类方法 [3,41,8,21,17]。为了公平比较，我们在同一块 A100 NVIDIA GPU 上报告了所有列在表 1中的模型的 GFLOPS 和 FPS。除 DETR 和 DAB-DETR 外，所有方法都使用多尺度特征。对于没有多尺度特征的方法，我们报告了使用 ResNet-DC5 的结果，其由于采用了扩展的大分辨率特征图而表现更好。由于一些方法采用 5 尺度的特征图，一些采用 4 ，我们报告了使用 4 和 5 尺度的特征图的结果。

[3] 我们在模型中使用了规范化的框形式，因此每个框的值是一个介于 0 和 1 之间的浮点数。给定两个框，我们在反 Sigmoid 变换后对它们进行求和，然后通过 Sigmoid 对求和结果进行变换。更多细节请参见 Deformable DETR [41] Sec. A.3。

Model	Epochs	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L	GFLOPS	Params	FPS
Faster-RCNN(5scale) [30]	12	37.9	58.8	41.1	22.4	41.1	49.1	207	40M	21*
DETR(DC5) [3]	12	15.5	29.4	14.5	4.3	15.1	26.7	225	41M	20
Deformable DETR(4scale)[41]	12	41.1	–	–	–	–		196	40M	24
DAB-DETR(DC5)† [21]	12	38.0	60.3	39.8	19.2	40.9	55.4	256	44M	17
Dynamic DETR(5scale) [8]	12	42.9	61.0	46.3	24.6	44.9	54.4	–	58M	–
Dynamic Head(5scale) [7]	12	43.0	60.7	46.8	24.7	46.4	53.9	–	–	–
HTC(5scale) [4]	12	42.3	–	–	–	–	–	441	80M	5*
DN-Deformable-DETR(4scale)† [17]	12	43.4	61.9	47.2	24.8	46.8	59.4	265	48M	23
DINO-4scale†	12	**49.0**(+5.6)	**66.6**	**53.5**	**32.0**(+7.2)	**52.3**	**63.0**	279	47M	24
DINO-5scale†	12	**49.4**(+6.0)	**66.9**	**53.8**	**32.3**(+7.5)	**52.5**	**63.9**	860	47M	10

表 1. DINO 和其他检测模型在 COCO val2017 上使用 ResNet50 主干网络训练了 12 轮次（即所谓的 1× 情景）。对于没有多尺度特征的模型，我们测试了它们的最佳模型 ResNet-50-DC5 的 GFLOPS 和 FPS。DINO 使用 900 个查询。† 表示使用 900 个查询或 300 个查询结合 3 模式的模型，这种模式与 900 个查询的效果相似。其他 DETR 类模型（除 DETR（100 个查询）外）使用 300 个查询。* 表示它们是使用 mmdetection [5] 框架进行测试的。

如表 1 所示，在相同情景下，我们的方法在使用 ResNet-50 和 4 -尺度特征图时获得了 +5.6 AP 的提升，在使用 5 -尺度特征图时获得了 +6.0 AP 的提升。我们的 4 -尺度模型在计算量和参数数量上没有引入太多开销。此外，我们的方法在小物体上的表现尤其出色，分别在 4 -尺度和 5 -尺度下获得了 +7.2 AP 和 +7.5 AP 的提升。需要注意的是，由于工程技巧，我们使用 ResNet-50 主干网络的模型结果比论文第一版中的结果更高。

与使用 ResNet-50 骨干网络的最佳模型比较： 为验证我们方法在提高收

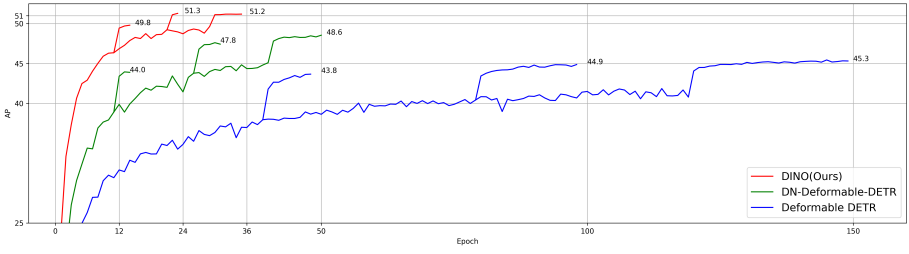

图 7. 训练收敛曲线在 COCO val2017 上评估，用于 DINO 和使用多尺度特征的两种先前最先进的模型。

敛速度和性能方面的有效性，我们使用相同的 ResNet-50 主干网络与几种强

Model	Epochs	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Faster-RCNN [30]	108	42.0	62.4	44.2	20.5	45.8	61.1
DETR(DC5) [41]	500	43.3	63.1	45.9	22.5	47.3	61.1
Deformable DETR [41]	50	46.2	65.2	50.0	28.8	49.2	61.7
SMCA-R [11]	50	43.7	63.6	47.2	24.2	47.0	60.4
TSP-RCNN-R [34]	96	45.0	64.5	49.6	29.7	47.7	58.0
Dynamic DETR(5scale) [7]	50	47.2	65.9	51.1	28.6	49.3	59.1
DAB-Deformable-DETR [21]	50	46.9	66.0	50.8	30.1	50.4	62.5
DN-Deformable-DETR [17]	50	48.6	67.4	52.7	31.0	52.0	63.7
DINO-4scale	24	**50.4**(+1.8)	68.3	54.8	33.3	53.7	64.8
DINO-5scale	24	**51.3**(+2.7)	69.1	56.0	34.5	54.2	65.8
DINO-4scale	36	**50.9**(+2.3)	69.0	55.3	34.6	54.1	64.6
DINO-5scale	36	**51.2**(+2.6)	69.0	55.8	35.0	54.3	65.3

表 2. DINO 和其他检测模型在 COCO val2017 上的结果，使用 ResNet-50 作为模型，在更多轮次 (24 , 36 , 或更多) 下训练。

基准方法进行比较。尽管最常见的 50 -轮次情景，我们采用 24 (2×) 和 36 (3×) 轮次情景，因为我们的方法收敛更快，在 50 -轮次训练中仅产生较小的额外收益。表 2中的结果显示，仅使用 24 轮次，我们的方法在 4 和 5 尺度上分别实现了 +1.8 AP 和 +2.7 AP 的提升。此外，在 3× 情景下使用 36 轮次，提升增加到 +2.3 和 +2.6 AP，分别在 4 和 5 尺度上。详细的收敛曲线比较如图7所示。

4.3 与最先进的模型对比

为了与最先进结果进行比较，我们使用了公开可用的 SwinL [23] 主干网络，在 ImageNet-22K 上进行预训练。我们首先在 Objects365 [33] 数据集上对 DINO 进行预训练，然后在 COCO 上进行微调。如表 3 所示，DINO 在 COCO val2017 和 test-dev 上取得了 63.2 AP 和 63.3 AP 的最佳结果，这表明其在更大模型大小和数据大小上的强大可扩展性。请注意，表 3 中的所有先前最先进模型都不使用基于 Transformer 解码器的检测头（HTC++ [4] 和 DyHead [7]）。这是首次将端到端 Transformer 检测器作为最先进模

Method	Params	Backbone Pre-training Dataset	Detection Pre-training Dataset	Use Mask	End-to-end	val2017 (AP)		test-dev (AP)	
						w/o TTA	w/ TTA	w/o TTA	w/ TTA
SwinL [23]	284M	IN-22K-14M	O365	✓		57.1	58.0	57.7	58.7
DyHead [7]	≥ 284M	IN-22K-14M	Unknown*			–	58.4	–	60.6
Soft Teacher+SwinL [38]	284M	IN-22K-14M	O365	✓		60.1	60.7	–	61.3
GLIP [18]	≥ 284M	IN-22K-14M	FourODs [18],GoldG+ [18,15]			–	60.8	–	61.5
Florence-CoSwin-H[40]	≥ 637M	FLD-900M [40]	FLD-9M [40]			–	62.0	–	62.4
SwinV2-G [22]	3.0B	IN-22K-ext-70M [22]	O365	✓		61.9	62.5	–	63.1
DINO-SwinL(Ours)	**218M**	IN-22K-14M	O365		✓	**63.1**	**63.2**	**63.2**	**63.3**

表 3. MS-COCO 上的最佳检测模型比较。与 DETR [3] 类似，我们使用“端到端”一词来表示模型是否摆脱了像 RPN 和 NMS 这样的手工设计组件。“使用掩码”一词表示模型是否使用实例分割标注进行训练。我们使用“IN”和“O365”分别表示 ImageNet [9] 和 Objects365 [33] 数据集。请注意，“O365”是“FourODs”和“FLD-9M”的子集。* DyHead 并未公开模型预训练所使用的数据集细节。

型建立在排行榜 [1] 上。与之前的最先进模型相比，我们使用的模型大小要小得多（1/15 参数，与 SwinV2-G [22] 相比），主干网络预训练数据大小（1/60 张图像，与 Florence 相比），以及检测预训练数据大小（1/5 张图像，与 Florence 相比），同时取得了更好的结果。此外，我们报告的性能不包括测试时增强（TTA），这是一个干净的结果，没有额外的花哨之处。这些结果有效地展示了 DINO 相比传统检测器的优越检测性能。

4.4 消融

#Row	QS	CDN	LFT	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
1. DN-DETR [17]	No			43.4	61.9	47.2	24.8	46.8	59.4
2. Optimized DN-DETR	No			44.9	62.8	48.6	26.9	48.2	60.0
3. Strong baseline (Row2+pure query selection)	Pure			46.5	64.2	50.4	29.6	49.8	61.0
4. Row3+mixed query selection	Mixed			47.0	64.2	51.0	31.1	50.1	61.5
5. Row4+look forward twice	Mixed		✓	47.4	64.8	51.6	29.9	50.8	61.9
6. DINO (ours, Row5+contrastive DN)	Mixed	✓	✓	**47.9**	**65.3**	**52.1**	**31.2**	**50.9**	**61.9**

表 4. 所提出算法组件的消融比较。我们使用术语“QS”、“CDN”和“LFT”分别表示“查询选择”、“对比去噪训练”和“向前查看两次”。

新算法组件的有效性：为了验证我们提出的方法的有效性，我们按照第3.1节所述，使用优化后的 DN - DETR 和纯查询选择构建了一个强大的基准。

我们在强大的基准中纳入了所有的流水线优化和工程技术（见第4.1节和附录D）。强大基准的结果见表4第 3 行。我们还在表4第 2 行展示了没有来自可变形 DETR[41] 的纯查询选择的优化后的 DN - DETR 的结果。虽然我们的强大基准优于所有先前的模型，但我们在 DINO 中的三种新方法进一步显著提高了性能。

5 结论

在本文中，我们提出了一种强大的端到端 Transformer 检测器 DINO，该检测器具有对比降噪训练、混合查询选择和两次展望，显著提高了训练效率和最终检测性能。因此，DINO 在使用多尺度特征的 12 -轮次和 36 -轮次设置下，在 COCO val2017 上优于所有之前的基于 ResNet-50 的模型。受此改进的启发，我们进一步探索了在更大数据集上使用更强的主干网络训练 DINO，并在 COCO 2017 test-dev 上取得了新的最先进水平，63.3 AP。这一结果确立了 DETR 类模型作为主流检测框架，不仅因为其新颖的端到端检测优化，还因为其卓越的性能。

图 8. 左图是使用带有 DN 查询训练的模型的检测结果，右图是我们的方法的结果。在左图中，箭头指向的男孩有 3 个重复的边界框。为清晰起见，我们只显示类别 “person” 的边界框。

参考文献

1. Papers with code - coco test-dev benchmark (object detection).
2. Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.
3. Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European conference on computer vision*, pages 213–229. Springer, 2020.
4. Kai Chen, Jiangmiao Pang, Jiaqi Wang, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jianping Shi, Wanli Ouyang, et al. Hybrid task cascade for instance segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4974–4983, 2019.
5. Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, et al. Mmdetection: Open mmlab detection toolbox and benchmark. *arXiv preprint arXiv:1906.07155*, 2019.
6. Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
7. Xiyang Dai, Yinpeng Chen, Bin Xiao, Dongdong Chen, Mengchen Liu, Lu Yuan, and Lei Zhang. Dynamic head: Unifying object detection heads with attentions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7373–7382, 2021.
8. Xiyang Dai, Yinpeng Chen, Jianwei Yang, Pengchuan Zhang, Lu Yuan, and Lei Zhang. Dynamic detr: End-to-end object detection with dynamic attention. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2988–2997, October 2021.
9. Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
10. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
11. Peng Gao, Minghang Zheng, Xiaogang Wang, Jifeng Dai, and Hongsheng Li. Fast convergence of detr with spatially modulated co-attention. *arXiv preprint arXiv:2101.07448*, 2021.
12. Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. Yolox: Exceeding yolo series in 2021. *arXiv preprint arXiv:2107.08430*, 2021.

13. Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.

14. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

15. Aishwarya Kamath, Mannat Singh, Yann LeCun, Ishan Misra, Gabriel Synnaeve, and Nicolas Carion. Mdetr – modulated detection for end-to-end multi-modal understanding. *arXiv: Computer Vision and Pattern Recognition*, 2021.

16. Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

17. Feng Li, Hao Zhang, Shilong Liu, Jian Guo, Lionel M Ni, and Lei Zhang. Dn-detr: Accelerate detr training by introducing query denoising. *arXiv preprint arXiv:2203.01305*, 2022.

18. Liunian Harold Li, Pengchuan Zhang, Haotian Zhang, Jianwei Yang, Chunyuan Li, Yiwu Zhong, Lijuan Wang, Lu Yuan, Lei Zhang, Jenq-Neng Hwang, et al. Grounded language-image pre-training. *arXiv preprint arXiv:2112.03857*, 2021.

19. Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollar. Focal loss for dense object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(2):318–327, 2020.

20. Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.

21. Shilong Liu, Feng Li, Hao Zhang, Xiao Yang, Xianbiao Qi, Hang Su, Jun Zhu, and Lei Zhang. DAB-DETR: Dynamic anchor boxes are better queries for DETR. *arXiv preprint arXiv:2201.12329*, 2022.

22. Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, et al. Swin transformer v2: Scaling up capacity and resolution. *arXiv preprint arXiv:2111.09883*, 2021.

23. Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10012–10022, 2021.

24. Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

25. Depu Meng, Xiaokang Chen, Zejia Fan, Gang Zeng, Houqiang Li, Yuhui Yuan, Lei Sun, and Jingdong Wang. Conditional detr for fast training convergence. *arXiv preprint arXiv:2108.06152*, 2021.

26. Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training, 2018.

27. Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.

28. Joseph Redmon and Ali Farhadi. Yolo9000: better, faster, stronger. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7263–7271, 2017.

29. Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

30. Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, 28, 2015.

31. Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6):1137–1149, 2017.

32. Hamid Rezatofighi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 658–666, 2019.

33. Shuai Shao, Zeming Li, Tianyuan Zhang, Chao Peng, Gang Yu, Xiangyu Zhang, Jing Li, and Jian Sun. Objects365: A large-scale, high-quality dataset for object detection. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 8430–8439, 2019.

34. Zhiqing Sun, Shengcao Cao, Yiming Yang, and Kris Kitani. Rethinking transformer-based set prediction for object detection. *arXiv preprint arXiv:2011.10881*, 2020.

35. Zhi Tian, Chunhua Shen, Hao Chen, and Tong He. Fcos: Fully convolutional one-stage object detection. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9627–9636, 2019.

36. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.

37. Yingming Wang, Xiangyu Zhang, Tong Yang, and Jian Sun. Anchor detr: Query design for transformer-based detector. *arXiv preprint arXiv:2109.07107*, 2021.

38. Mengde Xu, Zheng Zhang, Han Hu, Jianfeng Wang, Lijuan Wang, Fangyun Wei, Xiang Bai, and Zicheng Liu. End-to-end semi-supervised object detection with soft teacher. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3060–3069, 2021.

39. Zhuyu Yao, Jiangbo Ai, Boxun Li, and Chi Zhang. Efficient detr: Improving end-to-end object detector with dense prior. *arXiv preprint arXiv:2104.01318*, 2021.

40. Lu Yuan, Dongdong Chen, Yi-Ling Chen, Noel Codella, Xiyang Dai, Jianfeng Gao, Houdong Hu, Xuedong Huang, Boxin Li, Chunyuan Li, et al. Florence: A new foundation model for computer vision. *arXiv preprint arXiv:2111.11432*, 2021.

41. Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable detr: Deformable transformers for end-to-end object detection. In *ICLR 2021: The Ninth International Conference on Learning Representations*, 2021.

A 测试时间增强（TTA）

我们旨在构建一个不受手工设计组件影响的端到端检测器。然而，为了与传统检测模型进行比较，我们还探索了在类似 DETR 的模型中使用 TTA。我们仅在使用 SwinL 主干的大模型中使用它。与传统检测器相比，我们的 TTA 并未获得令人鼓舞的提升，但我们希望我们的探索能为未来的研究提供一些见解。

我们采用多尺度测试和水平翻转作为 TTA。然而，我们方法中集成不同增强的方法与传统方法不同，传统方法通常会输出重复的框。在传统方法中，集成是通过首先收集所有增强的预测并按置信度得分排序。然后通过 NMS 或框投票找到并消除重复的框。之所以首先收集所有增强的预测，是因为重复的框不仅出现在不同的增强之间，也出现在一个增强内部。这种集成方法会降低我们方法的性能，因为 DETR 类方法不容易输出重复的框，因为基于集合的预测损失抑制了重复预测，而集成可能会错误地移除真实的正预测 [3]。为了解决这个问题，我们设计了一种一对一的集成方法。假设我们有 n 个增强 $Aug_0, Aug_1, ..., Aug_{n-1}$ ，其中 Aug_i 有预测 $\mathbf{O}^i$ 和一个预定义的超参数权重 w^i 。$\mathbf{O}^i = \left\{(b_0^i, l_0^i, s_0^i), (b_1^i, l_1^i, s_1^i), ..., (b_{m-1}^i, l_{m-1}^i, s_{m-1}^i)\right\}$ 其中 b_j^i, l_j^i 和 s_j^i 分别表示第 j 个边界框、标签和得分。我们让 Aug_0 成为最主要的增强，也就是最可靠的一个。对于 $\mathbf{O}^0$ 中的每个预测，我们从其他每个增强 $\mathbf{O}^1, ..., \mathbf{O}^{n-1}$ 的预测中选择具有最高 IOU 的预测，并确保 IOU 高于预定义的阈值。最后，我们通过加权平均对选定的框进行集成，如下所示

$$b = \frac{1}{\sum I^i} \sum_{i=o}^{n-1} I^i w^i s_{idx(i)}^i b_{idx(i)}^i \tag{3}$$

其中 $I^i = 1$ 在 $\mathbf{O}^i$ 中至少有一个框的 IOU 高于阈值时为 $I^i = 0$ ，否则为 $I^i = 0$ 。$idx(i)$ 表示 $\mathbf{O}^i$ 中选定框的索引。

B 训练效率

我们将在表 5 中提供基础模型的 GPU 内存和训练时间。所有结果均在 8 块 Nvidia A100 GPU 上进行，使用 ResNet-50 [14]。结果表明，我们的模型不仅在训练中有效，而且高效。

Model	Batch Size per GPU	Traning Time	GPU Mem.	Epoch	AP
Faster RCNN [30]*	8	$\sim$ 60min/ep	13GB	108	42.0
DETR [3]	8	$\sim$ 16min/ep	26GB	300	41.2
Deformable DETR [41]*	2	$\sim$ 55min/ep	16GB	50	45.4
DINO(Ours)	2	$\sim$ 55min/ep	16GB	**12**	**47.9**

表 5. 不同模型的训练效率，使用 ResNet-50 作为主干网络。所有模型均使用 8 块 Nvidia A100 GPU 进行训练。所有结果均由我们报告。* Faster RCNN 的结果是通过 mmdetection 框架测试的。* 我们在测试期间使用了原始的 Deformable DETR，没有采用两阶段和边界框优化。

# Encoder/Decoder	6/6	4/6	3/6	2/6	6/4	6/2	2/4	2/2
AP	47.4	46.2	45.8	45.4	46.0	44.4	44.1	41.2

表 6. 在 COCO val2017 上使用 ResNet-50 主干网络对编码器层和解码器层数量的消融实验。我们在此使用 12 轮次设置和 100 DN 查询，没有负例。

C 对我们的模型组件的进一步分析

对编码器和解码器层数的分析：我们还研究了编码器和解码器层数变化的影响。如表 6 所示，减少解码器层数会对性能产生更显著的影响。例如，使用相同数量的 6 编码器层，而将解码器层数从 6 减少到 2 会导致 3.0 的 AP 下降。这种性能下降是预期的，因为框是通过每一层解码器动态更新和优化以获得最终结果的。此外，我们还观察到，与其他类似 DETR 的模型如 Dynamic DETR [7] 相比，当将解码器层数减少到 2 时，其性能下降了 13.8 AP（29.1 与 42.9 相比），而 DINO 的性能下降要小得多。这是因为我们的混合查询选择方法将编码器中选择的框传递给解码器查询。因此，解码器查询得到了良好的初始化，并不深度依赖于解码器层的优化。

# Denoising query	100 CDN	1000 DN	200 DN	100 DN	50 DN	10 DN	No DN
AP	47.9	47.6	47.4	47.4	46.7	46.0	45.1

表 7. 消融实验在 COCO 验证集上使用 ResNet-50 主干网络的降噪查询数量。注意，100 CND 查询对包含 200 个查询，其中 100 个是正样本查询，100 个是负样本查询。

查询降噪分析：我们继续通过改变降噪查询的数量来研究查询降噪的影响。我们使用了优化的动态降噪组（详细内容见附录 D.1）。如表 7 所示，当使用的降噪查询少于 100 时，增加数量可以显著提升性能。然而，在 100 之后继续增加 DN 数量只会带来很小的额外提升，甚至可能性能下降。我们在附录 C 中还分析了编码器和解码器层数的影响。

D 更多实施细节

D.1 动态 DN 组

在 DN-DETR 中，一张图像中的所有 GT 对象（标签 + 框）被收集为一个 GT 组进行降噪。为了提高 DN 训练效率，在训练过程中使用图像中 GT 组的多个带噪声版本。在 DN-DETR 中，根据不同的模型大小，组的数量设置为五或十。由于 DETR 类模型采用小批量训练，每个批次中每张图像的 DN 查询总数被填充到批次中的最大值。考虑到 COCO 数据集中一张图像中的对象数量范围从 1 到 80，这种设计效率低下并导致内存消耗过多。为了解决这个问题，我们提出固定 DN 查询的数量，并根据每张图像的对象数量动态调整组的数量。

D.2 大规模模型预训练

Objects365 [33] 是一个大规模的检测数据集，包含超过 $1.7M$ 个标注图像用于训练和 $80,000$ 个标注图像用于验证。为了更高效地使用数据，我们选择前 $5,000$ 个来自 $80,000$ 个验证图像作为我们的验证集，并将其余的添加到训练中。我们在 Objects365 上对 DINO 进行 26 轮次预训练，使用 64 块 Nvidia A100 GPU，并在 COCO 上对模型进行 18 轮次微调，使用 16 块 Nvidia A100 GPU。每块 GPU 的本地批量大小为 1 张图像。在微调阶段，我们将图像尺寸放大到 $1.5\times$ （即最大尺寸 1200×2000 ）。这会为最终结果增加约 0.5 的 AP。为了减少 GPU 内存使用，我们在训练过程中利用了检查点 [6] 和混合精度 [26]。此外，我们为此大型模型使用了 1000 个 DN 查询。

D.3 其他实现细节

基本超参数。对于超参数，如 DN-DETR 中所述，我们使用一个 6-层 Transformer 编码器和一个 6-层 Transformer 解码器以及 256 作为隐藏特征维度。

我们将初始学习率（lr）设置为 1×10^{-4}，并采用一个简单的 lr 调度器，在 11-轮次、20-轮次和 30-轮次时通过乘以 0.1 降低学习率，分别适用于 12、24 和 36 的轮次情景，使用 RestNet50。我们使用 AdamW [16,24] 优化器，权重衰减为 1×10^{-4}，并在 Nvidia A100 GPU 上以批量大小 16 训练我们的模型。由于 DN-DETR [17] 采用了 300 个解码器查询和 3 个模式 [37]，我们使用 $300 \times 3 = 900$ 个解码器查询，计算代价相同。我们的 DINO 与 SwinL 的学习率调度方案见附录。

损失函数。 我们使用 L1 损失和 GIOU [32] 损失进行框回归和焦点损失 [19] 与 $\alpha = 0.25, \gamma = 2$ 进行分类。如 DETR [3] 所述，我们在每个解码器层之后添加辅助损失。类似于可变形 DETR [41]，我们在查询选择模块之后添加额外的中间损失，其组件与每个解码器层相同。我们使用与 DAB-DETR [21] 和 DN-DETR [17] 相同的损失系数，即 1.0 用于分类损失， 5.0 用于 L1 损失， 2.0 用于 GIOU 损失。

详细的模型组件。 我们还优化了 DAB-DETR [21] 和 DN-DETR [17] 中使用的检测流水线。遵循 DN-Deformable-DETR [17]，我们采用与 Deformable DETR [41] 相同的多尺度方法并使用可变形注意力。DN-DETR 在不同的解码器层中使用不同的预测头和非共享参数。此外，我们引入动态降噪组以提高降噪训练效率并减轻内存开销（见附录 D.1）。在这项工作中，我们发现使用共享预测头可以带来额外的性能提升。这同时也导致约一百万参数的减少。此外，我们发现 DAB-DETR 中使用的条件查询 [25] 不适合我们的模型，因此未将其包含在最终模型中。

训练增强 我们在训练期间使用相同的随机裁剪和缩放增强，如 DETR [3] 所示。例如，我们随机调整输入图像的大小，使其较短边在 480 和 800 像素之间，较长边最多为 1333 。对于使用 SwinL 的 DINO，我们使用默认设置对模型进行预训练，但使用 1.5× 更大的尺度（较短边在 720 和 1200 像素之间，较长边最多为 2000 像素）进行微调，以与排行榜上的模型进行比较 [1]。不使用任何其他技巧，我们在 val2017 上获得 63.1 的结果，在 test-dev 上获得 63.2 的结果，无需测试时增强（TTA）（见附录A），优于 SwinV2 [22] 取得的先前最先进结果 63.1 ，解决方案更加简洁。

多尺度情景。 对于我们的 4 -尺度模型，我们从主干网络的阶段 2 、3 和 4 提取特征，并通过下采样阶段 4 的输出添加一个额外的特征。主干网络阶段 1 的另一个特征图用于我们的 5 -尺度模型。对于超参数，我们设置 $\lambda_1 = 1.0$ 和 $\lambda_2 = 2.0$ 并使用 100 CDN 对，其中包含 100 个正查询和 100 个负查询。

D.4 详细超参数

我们将在表 8 中列出那些希望复现我们结果的超参数。

Item	Value
lr	0.0001
lr_backbone	1e-05
weight_decay	0.0001
clip_max_norm	0.1
pe_temperature	20
enc_layers	6
dec_layers	6
dim_feedforward	2048
hidden_dim	256
dropout	0.0
nheads	8
num_queries	900
enc_n_points	4
dec_n_points	4
transformer_activation	“relu”
batch_norm_type	“FrozenBatchNorm2d”
set_cost_class	2.0
set_cost_bbox	5.0
set_cost_giou	2.0
cls_loss_coef	1.0
bbox_loss_coef	5.0
giou_loss_coef	2.0
focal_alpha	0.25
dn_box_noise_scale	0.4
dn_label_noise_ratio	0.5

表 8. 我们的模型中使用的超参数。